NeuroShell IRE – Product Functions and User Classes and Characteristics

T. Hariprasad, MSc Cybersecurity Research Project R26-CS-018 (NeuroShell Core Framework - Component 01 IRE) Cyber Operations Research Group
MSc Cybersecurity Research Project R26-CS-018

Revision History

Version	Date	Author	Changes
1.0	2026-09-05	NeuroShell Core Team	Initial comprehensive SRS

Contents

I NeuroShell IRE – Product Functions and User Classes and Characteristics 3

II 1. Overview and Document Scope 3

III 2. Product Functions 3
III-A Functional Architecture . . . 3
III-B Core Understanding Functions (Pipeline Stages) . . . 3
III-C Safety and Security Functions . . . 4
III-C1 Adversarial Input Detection (M1) . . . 4
III-C2 Hallucination Detection (S5/S6/S6B combined) . . . 4
III-C3 Role-Based Access Control (M2 / M5) . . . 5
III-C4 Audit Logging . . . 5
III-C5 Escalation Queue (Human-in-the-Loop) . . . 5
III-D Intelligence and Quality Functions . . . 5
III-D1 Sub-Intent Classification . . . 5
III-D2 Secondary Intent Detection . . . 6
III-D3 Self-Consistency Voting and Uncertainty . . . 6
III-D4 Semantic Caching . . . 6
III-D5 Session Management . . . 6
III-D6 Explainability (XAI) . . . 6
III-E Contract Generation and Integration Functions . . . 6
III-E1 PlannerContract Generation . . . 6
III-E2 API Surface . . . 6
III-E3 Downstream Integration . . . 6
III-E4 Operational Functions . . . 6

IV 3. User Classes and Characteristics 6
IV-A The RBAC Role Model . . . 6
IV-B Intent-by-Role Permission Matrix . . . 7
IV-C Role Characteristics in Detail . . . 7
IV-C1 viewer . . . 7
IV-C2 analyst . . . 7
IV-C3 operator . . . 7
IV-C4 admin . . . 8
IV-D Behavioural Rules and Defaults . . . 8
IV-E Machine Users and Human-in-the-Loop Classes . . . 8
IV-F Typical User Contexts . . . 8

V 4. Traceability and Effectiveness 9
V-A Function-to-Module-to-Test Traceability . . . 9
V-B Measured Functional Effectiveness . . . 9

VI 5. Summary 9

VII Appendix A – Enforcement Policy (config/enforcement_policy.py) 9

I. NeuroShell IRE – Product Functions and User Classes and Characteristics

Full technical report – Component 01 (Intent Recognition Engine) of the NeuroShell Core Framework

- **Project:** NeuroShell Core Framework, MSc Cybersecurity Research Project R26-CS-018
- **Component:** C1 – Intent Recognition Engine (IRE)
- **Report scope:** SRS sections 2.2 (Product Functions) and 2.3 (User Classes and Characteristics), expanded to implementation level
- **Sources:** `SRS_NeuroShell_IRE.md`, `IRE-Dev-Report.md`, `NeuroShell_IRE_Test_New_Results.md`, README, and source code under `src/`, `config/`, `data/`
- **Format legend:** rows marked "Module" reference the implementation file under `src/` or `config/`

II. 1. Overview and Document Scope

The IRE is a domain-specific Natural Language Understanding (NLU) gateway that converts natural-language offensive-security commands into validated, structured JSON intent contracts. This report expands the two SRS summary tables – Product Functions (2.2) and User Classes and Characteristics (2.3) – into a full implementation-grounded description.

The report is organised into four parts:

- **Part A: Product Functions** – the complete functional inventory of the IRE, grouped into core understanding, safety and security, intelligence and quality, and system and integration functions, each mapped to its implementing module.
- **Part B: User Classes and Characteristics** – the RBAC role model, role-by-role characteristics, intent-permission matrix, behavioural rules, and the operational personas and machine consumers the product serves.
- **Part C: Traceability and Effectiveness** – function-to-module-to-test traceability and measured functional effectiveness from the 900-record ablation study.
- **Appendix A** – the hallucination-taxonomy enforcement policy in full.

III. 2. Product Functions

A. Functional Architecture

Every request is processed by a deterministic seven-stage core wrapped by five middleware guards. Middleware runs around the core (before, after, or in parallel); the stages run in strict sequence.

```
Operator command (POST /parse, role + session context)
        v
M1 AdversarialDetector      Layer A lexical, Layer B structural, Layer C scoring
        v
M2 RBAC pre-inference check Fast-fail for restricted roles
        v
M3 SessionContext           Multi-turn context injection
        v
M4 SemanticCache            Exact-hash and cosine-similarity lookup
        v
S1 InputNormalizer          NFKC, whitespace, slang standardization
        v
S2 AliasResolver            Alias to canonical CVE and tool-term mapping
        v
S3 OllamaInference          Local LLM, few-shot prompt, strict JSON directive
        v
S4 JSONParser               Extraction, thinking-tag and markdown stripping
        v
S5 SchemaValidator          Pydantic IntentSchema constraint enforcement
        v
S6 RegexValidator           IP / CIDR / CVE / port / metachar verification
        v
S6B NetworkValidator        Network architecture and engagement-scope validation
        v
S7 ScopeGuard               RFC-1918 enforcement, warn-block modes, injection gate
        v
M5 RBAC post-inference check Intent-role policy matrix
        v
ContractBuilder             PlannerContract generation, tool and command synthesis
```

B. Core Understanding Functions (Pipeline Stages)

S1 Input Normalization. Applies NFKC Unicode normalisation, trims whitespace and punctuation artefacts, folds case, and standardises short passes such as expanded casing or slang. Produces a clean canonical token stream for the alias resolver and the LLM prompt.

TABLE I
TABLE 1.

#	Function	Module	Primary responsibility
S1	Input Normalization	`preprocessing/input_normalizer.py`	NFKC canonicalisation, whitespace collapsing, case folding, slang and filler standardization
S2	Alias Resolution	`preprocessing/alias_resolver.py`	Maps 22 offensive-security aliases to canonical identifiers (for example EternalBlue to CVE-2017-0144)
S3	LLM Inference	`inference/ollama_inference_engine.py`	Local Ollama inference (gemma4:latest default, qwen2.5-coder:7b alternative), few-shot prompt, temperature 0.1, strict JSON generation
S4	JSON Extraction	`validation/json_parser.py`	Extracts JSON from LLM output, strips markdown fences and think blocks
S5	Schema Validation	`validation/schema_validator.py`	Pydantic `IntentSchema` enforcement: intents, targets, ports, CVE format, confidence bounds
S6	Regex Validation	`validation/regex_validator.py`	Deterministic verification of IP, CIDR, CVE, port, URL and shell-metacharacter patterns
S6B	Network Architecture Validation	`validation/network_validator.py`	Validates target topology: CIDR width bounds (max prefix 30, i.e. no /8-style scans), sane subnets, architecture warnings
S7	Scope Guard Enforcement	`validation/scope_guard.py`	RFC-1918 and engagement-scope enforcement with strict / warn / disabled modes

S2 Alias Resolution. Consults `data/alias_map.json` (22 entries). Resolves exploit surnames (EternalBlue, Log4Shell, BlueKeep, ShellShock, Heartbleed, PrintNightmare, ZeroLogon, ProxyLogon, DirtyPipe, PwnKit), slang (pwn, box, brute, enum, recon) and tool terms (syn scan, ping sweep, stealthy sweep) into canonical CVEs and canonical action phrases before inference, grounding the prompt in known identifiers.

S3 LLM Inference. Runs a local model via Ollama with a strict-JSON directive prompt. Default temperature 0.1 with 2048 max tokens and a 60 s timeout. The model emits an intent candidate as JSON; no LLM output ever reaches the caller unvalidated.

S4 JSON Extraction. Recovers the JSON payload from LLM output by stripping code fences, markdown artefacts and thinking tags (thinking-tag stripping for Gemma chat templates). Emits a clean candidate object for schema validation.

S5 Schema Validation. Enforces the `IntentSchema` contract: exactly one of the 9 `IntentType` values, a `TargetType` with matched value, ports in range 1 to 65535, CVE identifiers matching `CVE-YYYY-NNNN` plus 4 to 7 trailing digits, `confidence` in the closed interval 0.0 to 1.0, and the invariant that `AMBIGUOUS` requires confidence below 0.5 while `REJECTED` requires a `rejection_reason`. Extra fields are rejected (`extra=forbid`).

S6 Regex Validation. Deterministically re-verifies every entity the LLM produced: IPv4 addresses and CIDR ranges, CVE syntax, port ranges, URL structure, and absence of shell metacharacter injection. This closes the loop on formats the model might emit even inside a valid JSON envelope.

S6B Network Architecture Validation. Applies engagement-scope and topology rules: targets must fall inside the configured engagement CIDR (default `192.168.0.0/16`); the widest allowed scan prefix is 30 (a /16 is the largest scan permitted), the narrowest is 32; malformed topologies and type/value mismatches fail here. Generates `ARCH_WARNING` and `SCOPE_WARNING` entries surfaced to the operator and the downstream planner.

S7 Scope Guard. Final enforcement gate: blocks or warns on engagement-scope violations depending on `engagement_mode` (strict, warn, disabled) and `scope_mode` (research, strict, production). Also serves as the final injection gate for adversarial input escalation.

C. Safety and Security Functions

1) Adversarial Input Detection (M1): Module `middleware/adversarial_detector.py`. A three-layer detector classifies every input before inference. Signals carry weights; the cumulative threat score is capped at 1.0.

Detection layers and key signal weights:

Rules: any signal with weight at or above 0.95 auto-blocks immediately (for example the explicit jailbreak keyword or a Gemma control-token injection). Otherwise the input is blocked when the cumulative threat score reaches the configured threshold (default 0.7). A dedicated special-character-density check (above 30%) and abnormal single-token length (above 100 characters) reinforce the structural layer.

2) Hallucination Detection (S5/S6/S6B combined): Module `validation/hallucination_taxonomy.py`. The pipeline classifies every failed validation into one of six `HallucinationClass` categories, and every check produces an audit-grade `ValidationFinding` (validator, passed, class, detail, severity).

TABLE II
TABLE 2.

Layer	Mechanism	Example signals	Weight range
A – Lexical	15 injection phrase patterns	ignore previous instructions, forget everything, DAN mode, role-prefix injection, Gemma / Llama / ChatML control tokens	0.8 to 1.0
B – Structural	anomaly patterns plus heuristics	hex/URL-encoded sequences, base64 payloads, script tags, template expressions, code-execution attempts, destructive commands	0.3 to 0.9
C – Scoring	combination logic	auto-block on any critical signal (weight 0.95+); else block when cumulative score reaches the threshold	threshold 0.7

The six hallucination classes and their enforcement modes (from `config/enforcement_policy.py`):

TABLE III
TABLE 3.

HallucinationClass	Detection locus	Enforcement	Reasoning
FABRICATED_CVE	Schema + Regex (CVE syntax)	block	Malformed CVE syntax indicates complete LLM hallucination
TARGET_TYPE_MISMATCH	Schema + Network	block	Structural mismatch between target type and value blocks downstream execution
CONTRADICTORY_ACTION_TARGET	Schema + Semantic policy	block	Incompatible action-target pairings or missing intent violate safety invariants
FABRICATED_PARAMETER	Regex + Schema	block	Out-of-bounds parameters or shell metacharacter injection present immediate execution risk
UNGROUNDED_CONFIDENCE	CVE grounding + semantic policy	warn	Syntactically valid but unindexed CVEs may be novel external vulnerabilities; advisory only
OUT_OF_SCOPE_TARGET	Network + Scope guard	warn / block	Warned in research mode, blocked in strict production mode

3) Role-Based Access Control (M2 / M5): Module `config/rbac_policy.py` and `middleware/rbac_guard.py`. See Part B for the full role model. Functionally, RBAC is enforced in two phases: a pre-inference fast-fail (M2) rejects restricted roles before any model cost, and a post-inference check (M5) validates the detected intent against the role matrix. Three intents are classed as sensitive (EXPLOITATION, PASSWORD_ATTACK, DIRECTORY_BRUTEFORCE) and are operator-plus only.

4) Audit Logging: Module `audit/audit_logger.py`. Every parse produces an append-only audit record persisted to `data/audit_log.db` (SQLite). Operator content is privacy-preserved via SHA-256 hashing while keeping intent, target type, role, latency and outcome as structured fields for later review.

5) Escalation Queue (Human-in-the-Loop): Admin-facing functions route high-severity findings to human review with an HTTP 202 pattern: `GET /admin/escalations`, `POST /admin/escalations/{audit_id}/resolve`. Explanatory and exploit-relevant intents require explicit human approval before execution.

D. Intelligence and Quality Functions

1) Sub-Intent Classification: Module `middleware/sub_intent_classifier.py`. Refines the top-level intent into one of 26 hierarchical sub-intents that drive tool selection in Component 02:

- **NETWORK_SCAN (6):** ICMP_DISCOVERY, SYN_STEALTH, UDP_SWEEP, VERSION_DETECT, OS_DETECT, FULL_PORT
- **EXPLOITATION (4):** REMOTE_CODE_EXEC, PRIV_ESC, LATERAL_MOVEMENT, PERSISTENCE
- **VULNERABILITY_AUDIT (3):** CVE_SPECIFIC, SERVICE_SPECIFIC, FULL_SCAN
- **SERVICE_ENUMERATION (4):** BANNER_GRAB, VERSION_DETECT, SMB, SNMP
- **PASSWORD_ATTACK (3):** BRUTE_FORCE, SPRAY, CREDENTIAL_STUFFING
- **PASSIVE_RECON (3):** DNS, WHOIS, OSINT
- **DIRECTORY_BRUTEFORCE (3):** WEB, API, FILES

For example `NETWORK_SCAN.SYN_STEALTH` maps to `nmap -sS`; `SERVICE_ENUMERATION.SMB` maps to `enum4linux`.

2) Secondary Intent Detection: Compound commands can carry a `secondary_intents` list (intent plus confidence, optionally with target). The response reports both the primary intent and the secondary context for downstream orchestration.

3) Self-Consistency Voting and Uncertainty: A two-tier scheme. Tier 1 runs a single low-temperature pass (0.1). When its confidence falls below the trigger threshold (default 0.7), Tier 2 re-samples N=5 candidates at temperature 0.7 and performs majority/consensus voting. Semantic entropy is computed over the candidate set and mapped into an uncertainty band: low below 0.3, high above 0.8. Outputs carry `uncertainty_band`, `raw_entropy`, `tier_triggered`, `resolution_method`, and `calibration_method`; ambiguous results can surface a `clarification_request`.

4) Semantic Caching: Module `middleware/semantic_cache.py`. Two lookup channels: an exact-hash key compare and a cosine-similarity match over embeddings of the request. The cache is invalidated when the CVE-grounding hash changes (a reindexed CVE database must not serve stale responses), and evicted LRU-style under the configured size limit. Cache hits short-circuit the pipeline, cutting p50 latency from seconds to tens of milliseconds (measured ~{}380x speedup per hit).

5) Session Management: Module `middleware/session_context.py`. Per-session turn tracking with a time-to-live expiry, injection of the last N turns (configurable memory window) into the inference prompt, last-target carry-over for anaphoric follow-ups, and session introspection and deletion via the API.

6) Explainability (XAI): Optional token-level attribution (integrated-gradients style) with a decision path, enabled through the `X-IRE-Explain` header; returned as an `XAIBlock` with `top_tokens` and `decision_path`.

E. Contract Generation and Integration Functions

1) PlannerContract Generation: Module `pipeline/contract_builder.py`. After validation, IRE synthesises a downstream-ready `PlannerContract` (`schemas/planner_contract.py`) with a strict, fully-validated field set: intent, sub_intent, validated target, ports, modifiers, CVE IDs, tool hint, schedule, confidence, rejection reason, scope warnings and session ID. The contract also carries a human-readable description, a priority level and a generated suggested command.

Priority determination: SCHEDULED when a cron schedule exists; CRITICAL for exploitation; HIGH for CVE-specific vulnerability audits; MEDIUM for scans, enumeration and generic audits; LOW otherwise.

Tool synthesis (intent to tool):

TABLE IV
TABLE 4.

Intent	Suggested tool
NETWORK_SCAN	nmap
VULNERABILITY_AUDIT	nmap (nikto when target is a URL)
SERVICE_ENUMERATION	nmap
DIRECTORY_BRUTEFORCE	gobuster
PASSWORD_ATTACK	manual (requires human approval)
EXPLOITATION	manual (requires human approval)
PASSIVE_RECON	none
AMBIGUOUS	none
REJECTED	none

Modifier-to-flag mapping translates semantic modifiers (stealth, aggressive, version detection, OS detection, udp and others) into tool flags for nmap and gobuster. CVE context and safety notes (for example the requirement of human approval for exploitation) are attached to the contract.

2) API Surface:

3) Downstream Integration: Outbound, non-blocking by default: `POST /parse` dispatches to the Dynamic Planner (Component 02, port 8002) via a background task when enabled (`planner_enabled`). `/execute` performs the synchronous chain and persists a chain cache so identical inputs short-circuit. `integration/planner_client.py` and `integration/executor_client.py` implement the authenticated service clients with API keys. All downstream components are optional and feature-flagged.

4) Operational Functions: `metrics_collector.py` records latency percentiles, error counts by stage and intent distribution. Rate limiting (30 requests per minute per IP) protects the LLM backend. Feature flags (`config/features.yaml`, `feature_flags.py`) toggle adversarial detection, RBAC, caching and downstream push without redeploys. The engagement scope and model family are administratively reconfigurable at runtime.

IV. 3. User Classes and Characteristics

A. The RBAC Role Model

IRE defines four privilege tiers in a strict hierarchy. Every request carries a `role` (schema field, default `analyst`). Enforcement is applied twice: once at the pre-inference gate and again after the intent is known.

Unknown roles are treated as `viewer`, the most restrictive class, so a bad role value can never grant capability.

TABLE V
TABLE 5.

Endpoint	Method	Function
/health	GET	Health and readiness (status, Ollama connectivity, model, cache size)
/parse	POST	Core parse: natural-language command to IREResponse V2
/execute	POST	Synchronous C1-to-C2 chain (parse then forward to planner), returns both outputs
/metrics	GET	Pipeline performance metrics (latency percentiles, error counts, intent distribution, uptime)
/session/{session_id}	GET / DELETE	Session introspection and clearing
/admin/rbac/{role}	GET	Role summary (permitted intents, RBAC state)
/admin/features	GET / POST /reload	Feature-flag inspection and hot reload
/admin/models	GET / POST /switch	Model listing and runtime model switching
/admin/enforcement-policy	GET	Live enforcement policy summary
/admin/escalations	GET / POST resolve	Human-in-the-loop escalation queue
/admin/audit/summary	GET	Audit trail summary
/admin/chain-cache	GET / DELETE	Persistent chain-cache inspection and flush
/engagement/scope	GET	Engagement scope configuration

TABLE VI
TABLE 6.

Role	Hierarchy	Privilege character	Read-only?	Typical persona
viewer	0 (least)	Observation only; pre-inference fast-fail	Yes	Auditor, board reviewer, passive observer
analyst	1	Passive recon, network scan and audit	Effectively	SOC analyst, security researcher, student
operator	2	All operations including exploitation and password attacks	No	Red-team operator, penetration tester
admin	3 (most)	Full: all intents plus model, policy and feature management	No	Platform administrator, team lead

B. Intent-by-Role Permission Matrix

Sensitive intents (EXPLOITATION, PASSWORD_ATTACK, DIRECTORY_BRUTEFORCE) are operator-plus; the three rows they occupy are the exact firewall between the analyst class and the operator class.

C. Role Characteristics in Detail

1) viewer:

- **Capabilities:** none for execution; the role exists so that observation and review channels are technically possible without granting action.
- **Restrictions:** pre-inference fast-fail. The request is rejected before any model inference or cache work, and every intent attempt is denied (`role in RESTRICTED_ROLES`).
- **Use case:** an auditor or reviewer who must be provably unable to cause actions.

2) analyst:

- **Default role.** When no role is supplied, the caller is treated as an analyst.
- **Capabilities:** NETWORK_SCAN, VULNERABILITY_AUDIT, SERVICE_ENUMERATION, PASSIVE_RECON, plus the meta-classes AMBIGUOUS and REJECTED.
- **Characteristics:** passive and observational operations only. An analyst can map a network, enumerate services, run a vulnerability audit and perform passive reconnaissance – everything a defensive/blue-team or research workflow needs – but cannot brute-force directories, exploit a host or attack credentials.
- **Typical engagement:** blue-team assessments, safe lab research, vulnerability discovery and reporting.

3) operator:

- **Capabilities:** all 9 intents, including the sensitive DIRECTORY_BRUTEFORCE, EXPLOITATION and PASSWORD_ATTACK intents.
- **Characteristics:** the active-operations class. An operator may commence exploitation flows and credential attacks, each of which is additionally gated by human approval (ContractBuilder marks these as requiring human approval, and the escalation queue provides the review path).
- **Typical engagement:** offensive red-team operations inside an authorised engagement boundary.

TABLE VII
TABLE 7.

Intent	viewer	analyst	operator	admin
NETWORK_SCAN	No	Yes	Yes	Yes
VULNERABILITY_AUDIT	No	Yes	Yes	Yes
SERVICE_ENUMERATION	No	Yes	Yes	Yes
PASSIVE_RECON	No	Yes	Yes	Yes
AMBIGUOUS	No	Yes	Yes	Yes
REJECTED	No	Yes	Yes	Yes
DIRECTORY_BRUTEFORCE	No	No	Yes	Yes
EXPLOITATION	No	No	Yes	Yes
PASSWORD_ATTACK	No	No	Yes	Yes

4) admin:

- **Capabilities:** all intents plus platform administration: model switching (POST /admin/models/switch), feature-flag reload, enforcement-policy inspection, escalation resolution and audit review.
- **Characteristics:** the trust boundary of the platform. Only admins reconfigure policy, so the enforcement matrix itself is protected from lower roles.
- **Typical engagement:** platform operation and governance of engagements.

D. Behavioural Rules and Defaults

TABLE VIII
TABLE 8.

Rule	Behaviour
Default role	`analyst` when no role is present
Unknown roles	Coerced to `viewer` (least privilege, implicit deny)
Restricted roles	`viewer` blocked pre-inference for any intent (fast-fail, no model cost)
Sensitive intents	EXPLOITATION, PASSWORD_ATTACK, DIRECTORY_BRUTEFORCE require operator or admin
Denied intent response	RBAC error names the denied intent and lists the caller's permitted intents
Engagement scope	All targets must fall within the configured engagement CIDR; mode strict/warn/disabled
Exploit / password flows	Require explicit human approval (escation-queue pattern)

E. Machine Users and Human-in-the-Loop Classes

The IRE serves two kinds of consumers beyond direct human operators:

Downstream machine consumers. Component 02 (Dynamic Planner) ingests `PlannerContract` objects over `POST /parse` push or `/execute`; Component 03 (AEERE executor) and Component 04 (AVAE vulnerability analysis) consume the validated downstream outcomes. These service-to-service users are trusted via shared API keys and feature-flagged integration, and they are strict contract consumers: they never receive free-form LLM output, only validated schemas.

Human-in-the-loop reviewers. The escalation queue defines a review class distinct from the four RBAC roles: reviewers and auditors who resolve high-severity findings (audit IDs), confirm or reject escalation, and inspect the append-only audit trail. In practice these are operators or admins with additional review duties, or external auditors using viewer-style observation.

F. Typical User Contexts

TABLE IX
TABLE 9.

Context	Primary role	Characteristic workflow
Security researcher / student lab	analyst	passive recon, scans, audits inside the lab CIDR
SOC analyst / blue team	analyst	continuous service enumeration and vulnerability audits
Penetration tester / red team	operator	scans then exploitation and credential workflows with approvals
Platform administrator	admin	policy, models, features, escalations, audit review
Governance / auditor	viewer	provably read-only observation and audit inspection

TABLE X
TABLE 10.

Function group	Primary modules	Test files
Input normalization	preprocessing/input_normalizer.py	tests/unit/test_input_normalizer.py
Alias resolution	preprocessing/alias_resolver.py	tests/unit/test_alias_resolver.py
JSON extraction	validation/json_parser.py	tests/unit/test_json_parser.py
Schema validation	validation/schema_validator.py	tests/unit/test_schema_validator.py
Regex validation	validation/regex_validator.py	tests/unit/test_regex_validator.py
Scope and injection gate	validation/scope_guard.py	tests/unit/test_scope_guard.py
Network architecture	validation/network_validator.py	tests/unit/test_network_validator.py
Semantic cache	middleware/semantic_cache.py	tests/unit/test_semantic_cache.py
Session context	middleware/session_context.py	tests/unit/test_session_context.py
Adversarial detection	middleware/adversarial_detector.py	tests/unit/test_adversarial_detector.py
Sub-intent classification	middleware/sub_intent_classifier.py	tests/unit/test_sub_intent_classifier.py
Feature flags	config/feature_flags.py	tests/unit/test_feature_flags.py
RBAC guard	middleware/rbac_guard.py	tests/unit/test_rbac_guard.py
Contract generation	pipeline/contract_builder.py	tests/unit/test_planner_contract.py
API and integration	api/main.py, integration/*	tests/integration/test_api.py + test_api_v2.py

V. 4. Traceability and Effectiveness

A. Function-to-Module-to-Test Traceability

Measured total: 482 tests (regex over test definitions, 2026-08-26), comprising unit and async integration tests across all 14 function groups above.

B. Measured Functional Effectiveness

Golden dataset: 900 records (400 well-formed + 500 deliberately flawed across eight categories); Exp3 ablation ran 8 configurations over all 900 records (7,200 pipeline runs).

TABLE XI
TABLE 11.

Configuration	Decision accuracy %	Flawed-input catch %	False-positive %
Full baseline pipeline	73.5	84	34
Validation stack removed	69.0	46	34
Schema + Regex removed	64.0	67	34
RBAC removed	79.5	69	34
Scope guard + adversarial removed	69.0	77	34
Semantic cache removed	71.0	84	34
Network validator removed	73.0	85	34
Tier-2 self-consistency removed	72.5	85	34

Interpretation for the function portfolio:

- The validation stack (S5, S6, S6B, S7) is the single largest safety contributor: removing it halves the catch rate (from 84% to 46%).
- Schema and Regex validators (S5, S6) are the most accuracy-critical functions (-9.5 pp accuracy when removed).
- RBAC looks like an accuracy cost (-6 pp with removal) but it deliberately blocks well-formed-but-unauthorised intent classes, which is why catch rate falls 15 pp when removed.
- Adversarial detection and scope enforcement contribute a combined 7 pp to catch rate.
- Semantic cache and network validator have negligible safety impact (they are latency and topology functions respectively).

Full per-class Exp4 results: adversarial injection caught at 95.6%, fabricated parameter 87.5%, target-type mismatch 81.2%, fabricated CVE 78.1%, contradictory action-target 59.4%, out-of-scope 46.7%, ambiguous 40%.

VI. 5. Summary

Product Functions: the IRE delivers twelve headline functions (intent classification, target and entity extraction, hallucination detection, adversarial detection, RBAC enforcement, session management, semantic caching, self-consistency voting, contract generation, audit logging, escalation queue) implemented as eight pipeline stages plus five middleware guards, exposed through fourteen REST endpoints, and validated by 482 tests.

User Classes and Characteristics: four RBAC roles in a strict hierarchy (viewer, analyst, operator, admin) with a 9-intent permission matrix, three sensitive intents, a default of analyst, and implicit-denial treatment of unknown roles – complemented by machine consumers (planner, executor, vulnerability analyzer) and human-in-the-loop reviewers in the escalation queue.

VII. Appendix A – Enforcement Policy (config/enforcement_policy.py)

TABLE XII
TABLE 12.

Class	Mode	Why
FABRICATED_PARAMETER	block	Out-of-bounds parameters or shell metacharacter injection present immediate execution risk
CONTRADICTORY_ACTION_TARGET	block	Incompatible action-target pairings or missing intent violate safety invariants
TARGET_TYPE_MISMATCH	block	Structural mismatch prevents downstream planner execution
FABRICATED_CVE	block	Malformed CVE syntax indicates complete LLM hallucination
UNGROUNDED_CONFIDENCE	warn	Valid syntax but unindexed CVE may be a novel external vulnerability
OUT_OF_SCOPE_TARGET	warn / block	Warned in research mode; blocked in strict production mode